Which SDTM Specification Artifacts Matter Most for AI-Assisted Code Generation?

Failure modes of AI-generated derivations, and the engineering artifacts that fix them

Rostislav Markov, Amazon Web Services

What was done

One AI model regenerated eight production SDTM derivation functions under three levels of engineering context: a prose description (**A1**), a behavior specification (**A2**) and a governed specification (**A3**). Every result was scored twice: was the logic right, and did the code fit the project framework?

Was the logic right?

functional correctness

A1 prose	A2 behavior spec	A3 governed spec
20%	84%	90%

Did it fit the framework?

integration quality

A1 prose	A2 behavior spec	A3 governed spec
0%	0%	82%

The fix is usually not a cleverer prompt. It is a reusable engineering artifact.

1 The question

AI can draft an SDTM derivation function in seconds. A regulated workflow still needs that function to be correct and to fit the project framework. Which engineering artifacts improve both, and where does prompting stop working?

2 Experiment

Eight production-derived SDTM functions

Function and what it derives	Complexity
`get_first_dose` earliest EX start date into the DM reference dates	low
`get_last_dose` latest EX end date into the DM reference dates	low
`get_last_contact` latest date across all domains into RFPENDTC	medium
`call_patient_death` death flag and date from DS and DD into DM	medium
`call_subjid` SUBJID copied from DC into the target domains	medium
`call_suppqual` SUPPQUAL and SQAP built from reference metadata	high
`create_epoch_taetord` EPOCH and TAETORD from SE, with tie-breaking	high
`create_se_domain` SE domain built from source domains via SQL	high

Three levels of engineering context

A1 — Function description

Brief prose only: the function name and a sentence of intent.

A2 — Behavioral specification

A1 plus Gherkin scenarios with concrete input rows, expected values, edge cases, exact error messages.

A3 — Governed specification

A2 plus project conventions, tests, types, helper interfaces.

Model fixed: Claude Sonnet 4.6 via Amazon Bedrock. Only the supplied context varied.

Two outcomes, scored separately

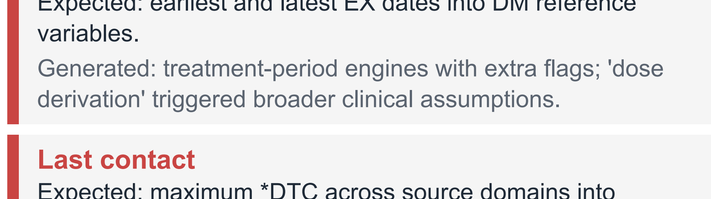
Functional correctness: was the logic right? Right algorithm, join keys, all target variables, validation rules with exact errors, edge cases, no side effects. Ten binary checks against the production code.

Integration quality: did it fit the framework? Imports, decorator and configuration, signature and return structure, test-harness compatibility. Three areas scored 0 to 3.

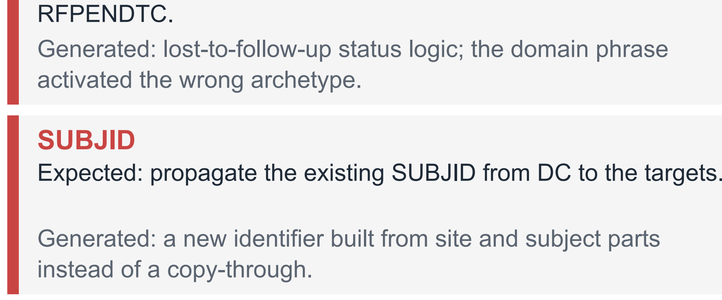
A function can be algorithmically correct and still fail the framework.

3 Results

Functional correctness *was the logic right?*



Integration quality *did it fit the framework?*



A3 reached 70% on the first round of project conventions and 82% after the third round (section 6).

Behavior examples fixed the logic. Conventions fixed the fit. The failure mode moved from semantic to mechanical.

4 Failure modes

Wrong logic at A1 (prose only): the model solved a plausible but wrong clinical problem

Dose dates
Expected: earliest and latest EX dates into DM reference variables.
Generated: treatment-period engines with extra flags; 'dose derivation' triggered broader clinical assumptions.

Last contact
Expected: maximum *DTC across source domains into RFPENDTC.
Generated: lost-to-follow-up status logic; the domain phrase activated the wrong archetype.

SUBJID
Expected: propagate the existing SUBJID from DC to the targets.
Generated: a new identifier built from site and subject parts instead of a copy-through.

Wrong fit at A1 and A2 (no project conventions): correct logic still failed the framework

- wrong imports
- missing decorator or configuration
- wrong signature and project types
- wrong return object
- approved helpers reimplemented instead of called
- missing test-harness compatibility

Behavior tells the model what to compute. Project contracts tell it how to fit.

5 Missing artifacts

Observed failure	Artifact the model lacked
Wrong logic	Behavior examples
Inconsistent style	Project conventions
Wrong imports	Dependency context
Wrong input structure	Metadata and schemas
Does not fit the system	Architecture context
Reinvented utilities	Capability registry

Failures are context gaps, not hallucinations.

6 Conventions help, then plateau

Three rounds of convention refinement at A3

Each round turned leftover deviations into new conventions.



Deviations left to fix: 19, then 7, then 6.

Fixed by conventions
imports, types, copying, return patterns

Resisted conventions
strong model defaults, archetype selection, exact wording

Not fixable from a prompt
cross-function dependencies, reference schemas, infrastructure patterns

7 Defaults beat instructions

The deviations that survived three rounds sat where the model has a strong default: constructing an identifier instead of copying one, or expanding a date derivation into a treatment-period engine.

What moved the outcome

Stronger instruction wording	negligible: 'avoid' matched 'MUST NOT'	follow-up study in progress
Strength of the model's default	dominant: resisted every wording	preliminary
Concrete behavior examples	large: correctness 20% to 84%	this study
Conventions, tests, helper contracts	large: integration 0% to 82%	this study

Override defaults with observable behavior and deterministic gates, not with authority words.

Three takeaways

1. **Better specifications matter more than prompt tricks.**
2. **Functional correctness and system integration are different problems.**
3. **Govern the artifacts that guide AI, not only the code it produces.**

From prompt engineering to specification engineering

Prompt → Structured prompt → Behavior specification → Validated specification → Reusable capabilities → Governed composition

AI may author; approved artifacts execute. Draft artifacts are not executable.

Poster, slides, paper and updates: github.com/rosmarkov/governed-sdtm-derivations

2026 CDISC US Interchange, Denver, Colorado

Related work: PharmaSUG 2026, paper AI-420, A specification-driven approach to improve the reliability of AI-generated SDTM transformation programs. PHUSE SDE West Windsor, July 2026, From AI-assisted coding to governed SDTM derivation development. Follow-up study on constraint compliance under strong pretraining priors: in preparation.